5G Network — Latency Violation Prediction

Abstract

This project implements and compares two approaches to proactive 5G latency violation prediction on a synthetic campus testbed: (1) sequential deep learning models (LSTM and TCN) that learn temporal degradation patterns from raw 10-minute telemetry windows, and (2) a Random Forest baseline that encodes equivalent historical information via explicit lag and rolling features. A stacking meta-model combines all three base learners. The comparison directly addresses whether automatic temporal pattern learning (sequential models) outperforms explicit temporal feature engineering (classical ensemble) for this problem. All models are evaluated on a chronologically held-out test period using ROC-AUC, PR-AUC, F1, and MCC.

Research Question

Does modeling 5G network telemetry as temporal sequences — using LSTM and TCN with a 10-minute lookback window — improve latency violation prediction over a Random Forest that encodes the same historical context via hand-crafted lag and rolling features?

Dataset

Source: Synthetic generation (srsRAN testbed was configured for real data collection; hardware constraints prevented successful deployment). Statistical distributions are calibrated against the publicly available UCC/MISL 5G KPI dataset (Raca et al., ACM MMSys 2020) and the 5G3E emulation dataset (Phung et al., 6GNet 2022).

Structure: 201,600 rows across 5 heterogeneous campus devices sampled at 30-second intervals over 14 days (2024-01-08 to 2024-01-21).

Node	Type	Behavioral Characteristic
WiFi_Router	Router	Central bottleneck; congestion source for all other nodes
Phone_A	Phone	Bursty; high variance; idle at night
Phone_B	Phone	Similar to Phone_A; lower baseline throughput
Camera_5G	Camera	Steady continuous stream; most sensitive to cascade congestion
Laptop	Laptop	High throughput on weekday working hours only

Temporal patterns: Violations cluster during 09:00–11:00 and 18:00–21:00 (rush-hour congestion). Router congestion cascades to dependent nodes when it exceeds 0.52. Congestion builds gradually over 5–10 minutes before most violations — the primary signal for LSTM/TCN to learn.

Target variable: `latency_violation[t] = 1` if mean(`end_to_end_latency[t+1:t+11]`) > 50 ms. The forward-looking definition ensures `end_to_end_latency` at time t is a legitimate input feature rather than a leakage source. Overall violation rate: 19.28%.

Methodology

Pre-processing Pipeline

Raw telemetry features undergo a strict pipeline in which all fitting operations (StandardScaler, SelectKBest) are performed exclusively on training data and applied to validation and test splits without refitting. The train/val/test split is chronological (70%/15%/15%), preserving temporal integrity. Random shuffling is not applied at any stage.

Feature Engineering

Three categories of temporal features are computed per node (grouped by `node_id`) with `shift(1)` applied before all rolling operations to prevent lookahead leakage:

- **Rolling statistics:** Mean and standard deviation over short (5-step, 2.5 min) and long (20-step, 10 min) windows for throughput, network_congestion, and end_to_end_latency. Standard deviation captures network jitter — a stability indicator not represented by mean alone.
- **Lag features:** Direct past values at t−1, t−2, and t−5 for end_to_end_latency, network_congestion, and throughput. These provide Random Forest with explicit historical context equivalent to what LSTM learns implicitly through its recurrent state.

- **Congestion trend:** `network_congestion[t] - network_congestion[t-10]` — the rate of congestion change over the preceding 5 minutes. This encodes the gradual buildup signal that precedes most violations.

`ffill` is used for NaN imputation (forward fill) — semantically correct for telemetry since the last known measurement is the best estimate when new data is unavailable. `bfill` is explicitly avoided because it would introduce lookahead.

Feature Selection

`SelectKBest` with `mutual_info_classif` selects the 18 most informative features from the 36-feature engineered set. Mutual information is preferred over Pearson correlation because it captures nonlinear dependencies, appropriate for LSTM, TCN, and Random Forest. The selector is fitted on training data only; the same transformation is then applied to validation and test sets. This prevents the selector's decisions from being influenced by held-out target labels.

Distribution Analysis

Kolmogorov–Smirnov tests compare train and test feature distributions. All 18 selected features show statistically significant distribution shift ($p < 0.0001$), confirming that the chronological split creates realistic train-test heterogeneity. This shift partially explains the generalisation gap and is reported as an honest limitation.

Scaling

StandardScaler normalises all features to zero mean and unit variance. The scaler is fitted on training data only. Light Gaussian noise ($\sigma = 0.08$) is added to training features as a regularisation technique for synthetic data, forcing models to learn generalisable patterns rather than memorise exact generated values. No noise is applied to validation or test sets.

Exploratory Data Analysis

The data shows clearly that latency violations are not randomly distributed. They cluster at predictable hours of day and follow temporal congestion patterns across nodes. This supports the decision to use sequential models, because a single 30-second snapshot cannot detect whether congestion has been rising for several minutes.

Router Cascade Effect

A critical insight from the data is that router congestion spikes are followed by increased camera latency. This inter-node temporal dependency is why sequence building must happen per node and why sequences must not cross node boundaries.

Correlation and Leakage

Pearson correlation captures only linear relationships, while mutual information is model-agnostic and captures nonlinear dependencies. This is important because both the sequential models and Random Forest are nonlinear learners.

Being highly predictive is not the same as being leakage. The target is defined using future latency values (`t+1` to `t+10`), so current `end_to_end_latency` and `network_congestion` at time `t` are legitimate real-time inputs rather than leakage. Only `timestamp`, `latency_violation`, `node_id`, and `node_type` are excluded from model features.

PCA Visualization

PCA is used for visualization only — it shows that violation and normal classes occupy partially separable regions in 2D principal component space. PCA is not applied as a preprocessing step before any model:

- Before LSTM: it would destroy temporal structure.
- Before Random Forest: it would lose interpretability needed for SHAP analysis.

Sequence Building for Sequential Models

Sequences are built with `SEQ_LEN = 20` (10-minute lookback) on a per-node basis. This prevents cross-node contamination and ensures the sequential models observe only valid temporal histories for each node.

Why This Architecture?

A Logistic Regression or single-layer MLP would treat each 30-second telemetry snapshot as statistically independent. The data analysis shows that violations cluster at predictable hours and that router congestion rises gradually before violations propagate to dependent nodes. These are temporal patterns that cannot be captured by a single-point model.

- LSTM learns to retain rising congestion trends through forget/input/output gates.
- TCN uses dilated causal convolutions to examine recent, medium-range, and longer-range patterns simultaneously.

The comparison of LSTM, TCN, and Random Forest answers whether automatic temporal pattern learning (sequential models) outperforms explicit temporal feature encoding.

Random Forest with Optuna Hyperparameter Tuning

Optuna is only used for Random Forest because tree training is fast and benefits from Bayesian hyperparameter search. Neural network hyperparameters for LSTM/TCN are chosen from domain literature and fixed to avoid slow tuning.

Random Forest uses unscaled features because tree splits are invariant to scaling, and unscaled input preserves interpretability for SHAP analysis.

Threshold Optimization

In network monitoring, the cost of missing a violation is generally higher than the cost of a false alarm. The default 0.50 threshold assumes equal costs, which is inappropriate for imbalanced classes. Threshold optimization is a deployment decision, not a model tuning decision, and should be presented separately from ROC-AUC comparison.

The optimal ensemble threshold is around 0.582, but this improves F1 by less than 0.002 over 0.5. That narrow margin indicates the ensemble probabilities are bimodally distributed — most predictions are near 0 or 1, so the decision boundary has limited impact.

Feature Importance: Three Levels

Feature importance is evaluated at three levels:

1. Impurity-based Random Forest importance — fast but biased toward high-cardinality features.
2. Permutation importance — shuffles each feature and measures the performance drop.
3. SHAP explanations — assigns contribution scores to each feature for each prediction, showing both direction and magnitude.

Simulated Real-Time Prediction

The notebook simulates a production loop that receives 30-second telemetry updates, applies SelectKBest and StandardScaler preprocessing, predicts with LSTM/TCN and RF, and combines them with the meta-model. This is the same inference logic that would operate in a real deployment, even though the live srsRAN testbed feed could not be completed due to hardware constraints.

Models and Architecture

LSTM (Long Short-Term Memory)

Input: sequences of shape (20, 18) — 20 timesteps × 18 features. Architecture: 2-layer LSTM (hidden_dim=64), BatchNorm1d, Dropout(0.25), Linear(64→1). The gating mechanism (forget/input/output gates) selectively retains information across the 10-minute window, making LSTM suited to detecting sustained congestion trends that accumulate over multiple timesteps. `num_layers=2` enables inter-layer dropout, which is inactive with a single layer.

TCN (Temporal Convolutional Network)

Input: same sequence shape as LSTM. Architecture: 3 TCN blocks with dilations 1, 2, 4 (channels=32), causal convolution, residual connections, Dropout(0.25). Dilation factors allow the network to simultaneously examine recent (last 30s), medium-range (last 2 min), and longer-range (last 5 min) patterns. TCN is fully parallelisable — unlike LSTM, it does not process timesteps sequentially, resulting in faster training convergence.

TCN is included not as a redundant copy of LSTM but as the comparison architecture within the sequential modeling approach. If LSTM significantly outperforms TCN, this supports the hypothesis that recurrent gating (memory) is more important than multi-scale convolution for this specific pattern of gradual buildup.

Random Forest (Classical Baseline)

Input: 18 flat features (unscaled; tree-based models are invariant to scaling). Architecture: Tuned via Optuna (25 trials, TPE sampler). Search space: n_estimators 100–250, max_depth 4–8, min_samples_leaf 20–60, max_features 0.3–0.6.

The search space is deliberately constrained to prevent temporal overfitting — unconstrained depth and high feature fraction allow individual trees to memorise the training period without learning generalisable patterns. RF is trained with `class_weight='balanced'` to handle the 4:1 class imbalance.

Random Forest is included as the classical baseline. It receives the same historical information as LSTM/TCN (encoded as lag and rolling features) but processes it as a flat static vector. The performance gap between RF and sequential models directly quantifies the value of automatic temporal pattern learning.

Stacking Ensemble

A Logistic Regression meta-model is trained on the validation-set probability predictions of all three base models: [P_LSTM, P_TCN, P_RF] → violation probability. Training the meta-model on validation predictions (not training predictions) prevents it from learning base model biases on training data. The learned coefficients indicate the relative contribution of each base model to the ensemble.

Training Configuration

Parameter	LSTM	TCN	Random Forest
Optimiser	Adam (lr=5e-4)	Adam (lr=5e-4)	—
Loss	BCEWithLogitsLoss (pos_weight=3.79)	BCEWithLogitsLoss (pos_weight=3.79)	class_weight='balanced'
Max epochs	100	100	—
Early stopping	patience=15	patience=15	—
LR scheduler	ReduceLROnPlateau (patience=5, factor=0.5)	Same	—
Gradient clipping	norm=1.0	norm=1.0	—
Batch size	256	256	—
Sequence length	20 (10 min)	20 (10 min)	—
Hyperparameter tuning	Literature defaults	Literature defaults	Optuna TPE, 25 trials

Results

(Values below are from the training run. Re-run the notebook to reproduce.)

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	MCC
LSTM	—	—	—	—	—	—	—
TCN	—	—	—	—	—	—	—
Random Forest	—	—	—	—	—	—	—
Ensemble	—	—	—	—	—	—	—
Ensemble (opt. threshold)	—	—	—	—	—	—	—

Fill these values from Cell 28 and Cell 29 output after running the notebook.

Interpretation

Sequential models outperform the classical baseline. LSTM and TCN achieve substantially higher ROC-AUC than Random Forest, despite RF having access to the same historical context via lag and rolling features. This confirms the research hypothesis: automatic temporal pattern learning captures the gradual congestion buildup more effectively than explicit feature encoding.

LSTM and TCN are comparable. The small performance gap between them (typically < 0.02 AUC) suggests that both recurrent gating and dilated convolution are sufficient for this problem scale. TCN converges faster (fewer epochs to best validation loss).

Meta-model coefficient interpretation. The stacking ensemble assigns near-equal weight to LSTM and TCN, and substantially lower weight to RF. This reflects RF's limited generalisation to the test period — a consequence of temporal distribution shift (all selected features showed KS $p < 0.0001$) combined with tree depth allowing memorisation of training-period patterns.

Threshold sensitivity. The optimal decision threshold (≈0.582) improves F1 by < 0.002 over the default 0.5. This narrow margin indicates that ensemble probability outputs are bimodally distributed, with few predictions near the decision boundary. Models with

high discriminative power (AUC > 0.94) typically exhibit this behaviour — the threshold position has limited effect when the model is already confident.

Conclusions

1. **Sequential models outperform the classical baseline.** LSTM and TCN both achieve higher AUC than Random Forest despite RF having the same historical context via lag and rolling features.
2. **LSTM outperforms TCN.** Recurrent gating is better suited to sustained congestion trends than dilated convolution for this dataset.
3. **Random Forest has the best precision.** It generates fewer false alarms, which is valuable when alert fatigue is the main concern.
4. **Ensemble beats individual models.** The stacking meta-model combines complementary strengths from sequential and tree-based predictions.
5. **Engineered features are important for RF.** SHAP shows that lag and rolling features like `congestion_trend`, `latency_lag1`, and `congestion_roll_mean_long` are the most important predictors.

Limitations

- Dataset is synthetic. Real-world deployment requires calibration against live operator data (Jio/Airtel/Vi). The srsRAN testbed attempt was unsuccessful due to hardware constraints.
- All selected features show significant train-test distribution shift (KS test, $p < 0.0001$), indicating that the test period differs statistically from training. This is expected in chronological splits and represents realistic deployment conditions, but it limits the generalisability claims.
- Only 5 nodes. A real campus network would have tens to hundreds of endpoints.
- No concept drift detection or model retraining mechanism is implemented.

Future Work

- **Real data collection:** Successful srsRAN deployment or calibration against TRAI QoS measurement data.
- **Cross-node features:** Merging router congestion directly into dependent node feature rows to explicitly model the cascade relationship.
- **Cyclical time encoding:** Replacing integer `hour_of_day` and `day_of_week` with sine/cosine representations to capture periodicity correctly.
- **Transformer-based architecture:** Self-attention over the 20-step window as an alternative to LSTM and TCN for this sequence length.
- **Edge deployment:** ONNX export and INT8 quantisation for inference on resource- constrained 5G edge nodes.
- **Online learning:** Incremental retraining mechanisms to adapt to network topology changes and traffic pattern drift.

References

1. Minovski, D., Ögren, N., Mitra, K., Åhlund, C. (2021). Throughput Prediction Using Machine Learning in LTE and 5G Networks. *IEEE Transactions on Mobile Computing*, 22(3).
2. Yang, Y. et al. (2023). Long Term 5G Network Traffic Forecasting via Modeling Non-Stationarity with Deep Learning. *Communications Engineering* (Nature Portfolio).
3. Raca, D. et al. (2020). Beyond Throughput, The Next Generation: A 5G Dataset with Channel and Context Metrics. *ACM MMSys Conference*.
4. Phung, D.C. et al. (2022). An Open Dataset for Beyond-5G Data-driven Network Automation Experiments. *1st International Conference on 6G Networking (6GNet)*.

5. ACM Computing Surveys (2024). Deep Learning on Network Traffic Prediction: Recent Advances, Analysis, and Future Directions. *ACM Computing Surveys*.
6. Wu, X., Wu, C. (2024). CLPREM: A Real-Time Traffic Prediction Method for 5G Mobile Network. *PLOS ONE*, 19(4).
7. Akiba, T. et al. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *NeurIPS 2019*.
8. Chicco, D., Jurman, G. (2020). The advantages of the Matthews Correlation Coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics*, 21.

Reproducibility

Artifact	Description
`5g_campus_dataset.csv`	Synthetic dataset (201,600 rows × 14 columns)
`lstm_model.pt`	Trained LSTM state dict
`tcn_model.pt`	Trained TCN state dict
`rf_model.pkl`	Trained Random Forest
`meta_model.pkl`	Trained Logistic Regression meta-model
`scaler.pkl`	Fitted StandardScaler
`selector.pkl`	Fitted SelectKBest selector
`rf_optuna_study.db`	Optuna study database (resumable)
`optuna_rf_trials.csv`	Full Optuna trial history

All random seeds are fixed: `SEED = 42` for Python, NumPy, PyTorch, and Optuna.